

DBMS

1. INTRODUCTION TO DBMS

1. Data

Data: Raw facts and figures that have no meaningful context by themselves.

Example: 85, Ravi, CSE

2. Database

Database: An organized collection of related data.

Example: Student database containing student ID, name, marks, etc.

3. DBMS

DBMS (Database Management System): Software used to create, store, manage, and retrieve data from a database.

Examples: MySQL, Oracle, PostgreSQL.

4. RDBMS

RDBMS (Relational DBMS): A DBMS that stores data in **tables (rows and columns)** and uses relationships between tables.

Examples: MySQL, Oracle, PostgreSQL.

5. File System vs DBMS

File System	DBMS
Data stored in files	Data stored in databases
More data redundancy	Less redundancy
Less security	Better security
Difficult data sharing	Easy data sharing

File System

DBMS

Difficult backup/recovery Better backup/recovery

Limited concurrency Supports concurrency

Remember:

File System → Simple but limited

DBMS → Organized, secure, and efficient

6. Advantages of DBMS

- Reduces data redundancy
 - Improves data security
 - Provides data consistency
 - Supports data sharing
 - Provides backup and recovery
 - Supports concurrent access
 - Maintains data integrity
-

7. Disadvantages of DBMS

- High cost
 - Complex to manage
 - Requires trained people
 - Uses more memory/resources
 - Failure can affect many users
-

8. DBMS Applications

DBMS is used in:

- Banking
- Railway/Airline reservation
- Education

- Hospitals
 - E-commerce
 - Telecommunications
 - Social media
-

9. DBMS Users

Main types:

- **DBA** – Manages the database
 - **Database designers** – Design database structure
 - **Application programmers** – Develop database applications
 - **End users** – Use the database
-

10. DBA

DBA (Database Administrator): Person responsible for managing and maintaining the database.

Main tasks:

- Security
 - Backup & recovery
 - Performance monitoring
 - User access control
 - Database maintenance
-

11. Schema

Schema: The logical structure/design of a database.

Example:

Student(ID, Name, Branch, Marks)

Remember:

Schema = Structure

12. Instance

Instance: The actual data stored in the database at a particular time.

Remember:

Schema = Structure

Instance = Data

13. Metadata

Metadata: Data that describes other data.

Example: Table name, column name, data type, size.

Remember:

Metadata = Data about data

14. Data Independence

Data Independence: Ability to change the database structure at one level without affecting the next higher level.

Two types:

A. Physical Data Independence

Ability to change the **physical storage** without changing the logical structure.

Example: Changing storage method/indexing.

B. Logical Data Independence

Ability to change the **logical structure** without changing user views.

Example: Adding a new column to a table.

Easy memory:

Physical → Storage changes

Logical → Structure changes

15. Three-Schema Architecture

DBMS has **3 levels**:

External Level



Conceptual Level



Internal Level

1. External Level

User view of the database.

Different users can have different views.

2. Conceptual Level

Overall logical structure of the database.

Defines tables, relationships, constraints, etc.

3. Internal Level

How data is physically stored.

Deals with files, indexes, storage, etc.

★ Quick Revision

Data → Raw facts

Database → Organized collection of data

DBMS → Software to manage database

RDBMS → DBMS using tables and relationships

Schema → Structure

Instance → Actual data

Metadata → Data about data

Physical Data Independence → Storage changes

Logical Data Independence → Logical structure changes

External → User view

Conceptual → Logical database structure

Internal → Physical storage

DBA → Manages database

🔥 Placement Traps

Q: Schema or Instance changes frequently?

→ Instance

Q: Which level deals with physical storage?

→ Internal level

Q: Which level represents user views?

→ External level

Q: Which independence deals with storage changes?

→ Physical data independence

Q: Which independence is harder to achieve?

→ Logical data independence

2. DATABASE MODELS

1. Database Model

Database Model: A way of describing how data is organized and related in a database.

2. Hierarchical Model

Hierarchical Model: Stores data in a **tree-like structure** using parent-child relationships.

Example: Organization structure.

Company

└─ HR

└─ IT

└─ Developer

└─ Tester

Limitation: Mainly supports one-to-many relationships.

3. Network Model

Network Model: Stores data as a **graph of records** connected by relationships.

Advantage: Supports many-to-many relationships.

4. Relational Model

Relational Model: Stores data in **tables (rows and columns)**.

Example:

Student

```
+---+-----+-----+
| ID | Name | Marks |
+---+-----+-----+
```

Most commonly used model.

5. Object-Oriented Model

Object-Oriented Model: Stores data as **objects**, similar to object-oriented programming.

Supports:

- Objects
 - Classes
 - Inheritance
 - Encapsulation
-

6. NoSQL Model

NoSQL: Databases that don't primarily use the traditional relational table model.

Types:

- Key-Value
- Document

- Column
- Graph

Examples: MongoDB, Redis, Cassandra, Neo4j.

★ Relational vs NoSQL

Relational	NoSQL
Tables	Documents/Key-value/Graphs etc.
Fixed schema	Flexible schema
SQL	Database-specific/query APIs
Strong relationships	Designed for flexible/scalable data
MySQL, Oracle	MongoDB, Redis

🔥 Placement Revision

Hierarchical → Tree

Network → Graph/Records

Relational → Tables

Object-Oriented → Objects

NoSQL → Non-relational models

Important MCQ

Q: Which model stores data in rows and columns?

→ **Relational model**

Q: Which model uses parent-child relationships?

→ **Hierarchical model**

Q: Which model supports objects and inheritance?

→ **Object-oriented model**

Q: MongoDB belongs to which category?

→ **NoSQL database**